

```

#include <iostream>
#include <vector>
#include <omp.h>

using namespace std;

class Graph {
    int V;
    vector<vector<int>> adj;

public:
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // undirected graph
    }

    void printGraph() {
        cout << "\nAdjacency List:\n";
        for (int i = 0; i < V; i++) {
            cout << i << " -> ";
            for (int j : adj[i])
                cout << j << " ";
            cout << endl;
        }
    }

    // □ Parallel BFS
    void parallelBFS(int start) {
        vector<bool> visited(V, false);
        vector<int> current, next;

        visited[start] = true;
        current.push_back(start);

        cout << "\nParallel BFS Traversal: ";

        while (!current.empty()) {
            next.clear();

```

```

#pragma omp parallel
{
    vector<int> local_next;

    #pragma omp for nowait
    for (int i = 0; i < current.size(); i++) {
        int node = current[i];

        #pragma omp critical
        cout << node << " ";

        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                #pragma omp critical
                {
                    if (!visited[neighbor]) {
                        visited[neighbor] = true;
                        local_next.push_back(neighbor);
                    }
                }
            }
        }
    }

    #pragma omp critical
    next.insert(next.end(), local_next.begin(), local_next.end());
}

current = next;
}
cout << endl;
}

```

```

// □ Parallel DFS (Task-based)
void dfsUtil(int node, vector<bool> &visited) {
    bool alreadyVisited = false;

    // Protect access to visited[]
    #pragma omp critical
    {
        if (visited[node]) {
            alreadyVisited = true;
        } else {
            visited[node] = true;

```

```

        cout << node << " ";
    }
}

// Exit AFTER critical section
if (alreadyVisited) return;

// Create tasks for neighbors
for (int neighbor : adj[node]) {
    #pragma omp task
    dfsUtil(neighbor, visited);
}

#pragma omp taskwait
}

void parallelDFS(int start) {
    vector<bool> visited(V, false);

    cout << "\nParallel DFS Traversal: ";

    #pragma omp parallel
    {
        #pragma omp single
        {
            dfsUtil(start, visited);
        }
    }

    cout << endl;
}

};

int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;

    cout << "Enter each edge (u v) as space-separated integers:\n";

```

```

for (int i = 0; i < E; i++) {
    int u, v;
    cin >> u >> v;
    g.addEdge(u, v);
}

g.printGraph();

int start;
cout << "\nEnter starting vertex for BFS and DFS: ";
cin >> start;

g.parallelBFS(start);
g.parallelDFS(start);

return 0;
}

```

The image shows a terminal window titled "dbmslab@fedora:~/BE HPC". The user has compiled a C++ program named "HPC1.cpp" using "g++ -fopenmp HPC1.cpp -o HPC1" and then executed it with "./HPC1". The program prompts for the number of vertices (6) and edges (4), followed by the edges themselves (0 1, 1 2, 2 3, 3 4). It then displays the adjacency list and performs parallel BFS and DFS traversals starting from vertex 2. The BFS traversal order is 2 3 1 0 4, and the DFS traversal order is 2 1 3 4 0.

```

dbmslab@fedora:~/BE HPC
[dbmslab@fedora BE HPC]$ g++ -fopenmp HPC1.cpp -o HPC1
[dbmslab@fedora BE HPC]$ ./HPC1
Enter number of vertices: 6
Enter number of edges: 4
Enter each edge (u v) as space-separated integers:
0 1
1 2
2 3
3 4

Adjacency List:
0 -> 1
1 -> 0 2
2 -> 1 3
3 -> 2 4
4 -> 3
5 ->

Enter starting vertex for BFS and DFS: 2
Parallel BFS Traversal: 2 3 1 0 4
Parallel DFS Traversal: 2 1 3 4 0
[dbmslab@fedora BE HPC]$

```

```
[dbmslab@fedora BE HPC]$ g++ -fopenmp HPC_1.cpp -o HPC1
```

```
[dbmslab@fedora BE HPC]$ ./HPC1
```

```
Enter number of vertices: 6
```

```
Enter number of edges: 4
```

```
Enter each edge (u v) as space-separated integers:
```

```
0 1
```

```
1 2
```

```
2 3
```

```
3 4
```

```
Adjacency List:
```

```
0 -> 1
```

```
1 -> 0 2
```

```
2 -> 1 3
```

```
3 -> 2 4
```

```
4 -> 3
```

```
5 ->
```

```
Enter starting vertex for BFS and DFS: 2
```

```
Parallel BFS Traversal: 2 3 1 0 4
```

```
Parallel DFS Traversal: 2 1 3 4 0
```

```
[dbmslab@fedora BE HPC]$
```